NAME: Shaikh Areeba Mohammed Ismail
CLASS/BATCH: TE-B2
ROLL NO: 46
SUBJECT: Artificial Intelligence

GROUP: A

ASSIGNMENT NO: 03

Problem Statement: Implement Greedy search algorithm for any of the following application:

- Selection Sort
- Minimum Spanning Tree
- Single-Source Shortest Path Problem
- Job Scheduling Problem
- Prim's Minimal Spanning Tree Algorithm
- Kruskal's Minimal Spanning Tree Algorithm
- Dijkstra's Minimal Spanning Tree Algorithm

```
# ==============================
# PART 1: SELECTION SORT
# ==============================

# Selection Sort is a Greedy Algorithm because:
# At each step, it selects the smallest element from the
unsorted portion
# and places it at the correct position.

def selection_sort(arr):

    # Get the length of the list
    n = len(arr)

    # Outer loop controls the boundary between sorted and
unsorted parts
    for i in range(n):

        # Assume the current index is the minimum
        min_index = i

        # Inner loop finds the smallest element in remaining
unsorted array
        for j in range(i + 1, n):
```

```
            # Compare current element with current minimum
            if arr[j] < arr[min_index]:
                min_index = j    # Update index of smallest
element

        # Swap the found minimum element with the first
element
        arr[i], arr[min_index] = arr[min_index], arr[i]

        # Print step-by-step result
        print(f"Step {i+1}: {arr}")

    return arr

# Example usage
data = [64, 25, 12, 22, 11]
print("Original Array:", data)
sorted_array = selection_sort(data)
print("Sorted Array:", sorted_array)
```

OUTPUT:

```
Original Array: [64, 25, 12, 22, 11]
Step 1: [11, 25, 12, 22, 64]
Step 2: [11, 12, 25, 22, 64]
Step 3: [11, 12, 22, 25, 64]
Step 4: [11, 12, 22, 25, 64]
Step 5: [11, 12, 22, 25, 64]
Sorted Array: [11, 12, 22, 25, 64]


Selected Edge: ('A', 'B', 1)
Selected Edge: ('B', 'C', 2)
Selected Edge: ('B', 'D', 4)

Minimum Spanning Tree: [('A', 'B', 1), ('B', 'C', 2), ('B',
'D', 4)]
Total Weight of MST: 7
```

```python
# ========================================
# PART 2: MINIMUM SPANNING TREE (KRUSKAL)
# ========================================

# Kruskal's Algorithm is a Greedy Algorithm because:
# It always selects the smallest weight edge that does not
form a cycle.

# Disjoint Set (Union-Find) Data Structure
class DisjointSet:

    def __init__(self, vertices):
        # Parent dictionary stores parent of each vertex
        self.parent = {v: v for v in vertices}

        # Rank dictionary helps in union by rank optimization
        self.rank = {v: 0 for v in vertices}

    def find(self, vertex):
        # Find the root parent of the vertex
        if self.parent[vertex] != vertex:
            self.parent[vertex] =
self.find(self.parent[vertex])  # Path compression
        return self.parent[vertex]

    def union(self, v1, v2):
        # Find roots of both vertices
        root1 = self.find(v1)
        root2 = self.find(v2)

        # If roots are different, union them
        if root1 != root2:
            if self.rank[root1] > self.rank[root2]:
                self.parent[root2] = root1
            elif self.rank[root1] < self.rank[root2]:
                self.parent[root1] = root2
            else:
                self.parent[root2] = root1
                self.rank[root1] += 1


def kruskal_mst(vertices, edges):

    # Sort edges based on weight (Greedy choice)
    edges = sorted(edges, key=lambda x: x[2])

    # Create disjoint set
    ds = DisjointSet(vertices)
```

```
    mst = []   # List to store MST edges
    total_weight = 0

    for edge in edges:
        u, v, weight = edge

        # Check if including this edge forms a cycle
        if ds.find(u) != ds.find(v):
            ds.union(u, v)
            mst.append(edge)
            total_weight += weight
            print(f"Selected Edge: {edge}")

    return mst, total_weight

# Example Graph
vertices = ['A', 'B', 'C', 'D']
edges = [
    ('A', 'B', 1),
    ('A', 'C', 3),
    ('B', 'C', 2),
    ('B', 'D', 4),
    ('C', 'D', 5)
]

mst, weight = kruskal_mst(vertices, edges)

print("\nMinimum Spanning Tree:", mst)
print("Total Weight of MST:", weight)
```

OUTPUT:

```
Selected Edge: ('A', 'B', 1)
Selected Edge: ('B', 'C', 2)
Selected Edge: ('B', 'D', 4)

Minimum Spanning Tree: [('A', 'B', 1), ('B', 'C', 2), ('B',
'D', 4)]
Total Weight of MST: 7
```